

LIM M2

Processor Architecture Reference Manual

A deterministic, high-frequency 16-bit microprogrammed processor architecture featuring conflict-free multiplexer routing and 24-bit extended addressing.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

OFFICIAL REFERENCE SPECIFICATION

Revision 0.2 — October 2026

LIM M2 Core Architecture Manual. All rights reserved.

Contents

1	Document Scope and Architectural Philosophy	3
1.1	Notational Conventions	3
2	Processor Overview	3
2.1	Conceptual Datapath Schematic	5
3	Architecturally Visible Registers	6
3.1	Flag and Status Register (FREG) Mapping	6
4	Machine Instruction Format	6
5	Instruction Set Architecture Summary	7
6	Detailed Instruction Specifications	8
	NOP — No Operation and Arithmetic Chain Flush	9
	ADD — Addition with Automatic Multi-Word Carry Chaining	11
	SUB — Subtraction with Automatic Multi-Word Borrow Chaining	13
	MUL — Multiplication with Upper Word Residual Retention	15
	DIV — Division with Remainder Latched in ALU Residual	17
	LMR — Load Word from Memory to Register	18
	LRM — Store Register Word to Memory	19
	LRR — Low-Latency Register-to-Register Copy	20
	CMP — Hardware Operand Comparison	21
	JMP — Unconditional Program Branch	22
	JCR — Conditional Branch on Carry	23
	JNZ — Conditional Branch on Non-Zero	24
	JZ — Conditional Branch on Zero	25
	AND — Bitwise Logical AND	26
	OR — Bitwise Logical OR	27
	NAND — Bitwise Logical NAND	28
	NOR — Bitwise Logical NOR	29
	NOT — Bitwise Inversion (One's Complement)	30
	XOR — Bitwise Exclusive OR	31
	XNOR — Bitwise Equivalence	32
	INC — Atomic Increment by 1	33
	DEC — Atomic Decrement by 1	34
	PUSH — Push Word onto Execution Stack	35
	POP — Pop Word from Execution Stack	36
	BSL — Barrel Shift Left	37

BSR — Barrel Shift Right 38

CSRM — Control and Status Register Access 39

RSV27 — Architectural Extension Slot 40

RSV28 — Architectural Extension Slot 41

RSV29 — Architectural Extension Slot 42

RSV30 — Architectural Extension Slot 43

RSV31 — Architectural Extension Slot 44

7 Physical Package and Pinout (34-Pin Form Factor) 45

7.1 Pin Descriptions 45

8 Conclusion 45

1 Document Scope and Architectural Philosophy

This document serves as the formal architectural reference manual and hardware specification for the **LIM M2** microprocessor. The LIM M2 architecture has been deliberately designed to deliver deterministic execution, robust electrical integrity, and superior clock scaling.

A defining cornerstone of the LIM M2 datapath is the **total elimination of internal tri-state (high-impedance / Hi-Z) buses**. By substituting shared capacitive wires with actively driven multiplexer trees (Point-to-Point MUX Routing), the architecture prevents shoot-through currents, eliminates bus contention risks, and guarantees sharp, symmetrical transition edges.

Instruction Code Allocation

The LIM M2 instruction set architecture utilizes a contiguous 5-bit primary opcode space: NOP=0, ADD=1, ..., CSRM=26. Opcode values 27–31 are reserved for next-generation hardware extensions.

1.1 Notational Conventions

Notation	Architectural Meaning
dst	Destination register targeted for writeback
src, srcA, srcB	Source register operand identifiers
imm	Immediate constant scalar embedded within the instruction
addr	Memory address (16-bit primary or 24-bit extended pointer)
cfg	3-bit operational mode configuration field
ZF, CF, OF	Hardware status flags (Zero, Carry, and Overflow) within FREG

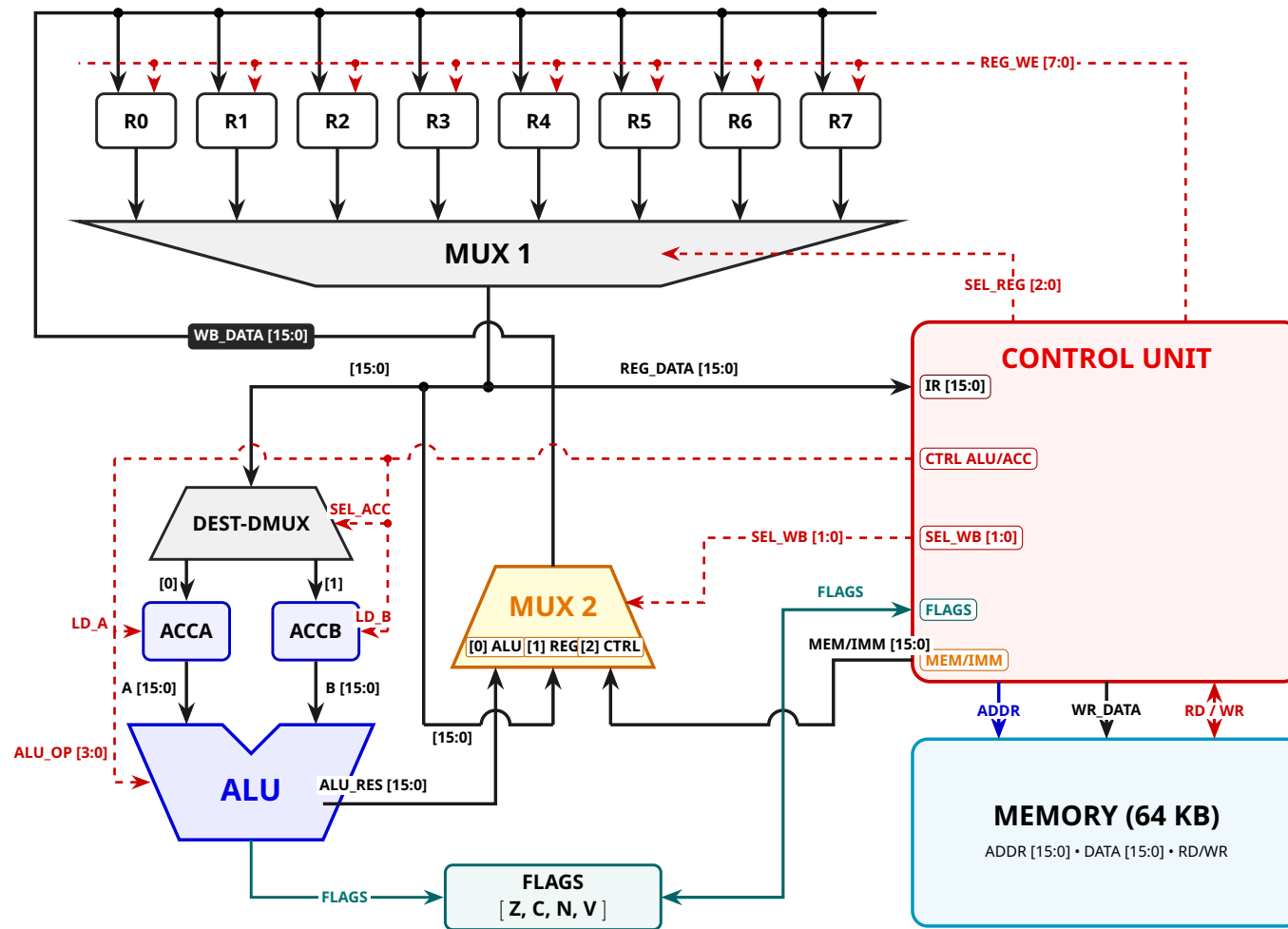
2 Processor Overview

The **LIM M2** is a versatile, high-throughput 16-bit microprogrammed processor engineered for integer-intensive computation. Architectural highlights include:

- **Conflict-Free Multiplexer Datapath:** Internal signals are steered purely by push-pull multiplexer trees (**MUX 1**, **DEST-DMUX**, and **MUX 2**). This modern design is perfectly suited for high-density ASIC and modern FPGA synthesis.
- **Dual Accumulator Staging:** Dedicated pre-ALU accumulators (ACCA and ACCB) isolate the arithmetic core from the general register file, minimizing propagation path skew.
- **Hardware ALU Auto-Chaining (Multi-Word Arithmetic):** The ALU integrates an internal residual register (ALU_RESIDUAL). When an addition overflows 16 bits ($> 2^{16} - 1$), a subtraction produces a borrow (< 0), or multiplication/division produces high-word residuals, the excess is retained in hardware and automatically accumulated or subtracted in the next mathematical operation. This enables continuous multi-word arithmetic (32, 48, 64+ bits) without dedicated ADC/SBB instructions. The chain is flushed using NOP.
- **Direct Reg-to-Reg Bypass:** A low-latency path from MUX 1 directly into MUX 2 enables single-cycle register transfers without locking ALU resources.
- **Linear 24-Bit Addressing (Up to 16 MB):** Dedicated 24-bit pointer registers (PTREG with auto-increment and SREG stack pointer) transcend the traditional 64 KB memory barrier, providing linear access across 16 megabytes of data and stack space.
- **High Clock Scaling Potential:** Unburdened by capacitive bus decay, the LIM M2 operates comfortably at a baseline frequency of **200 MHz**, scaling toward **700 MHz and above** in optimized

silicon.

2.1 Conceptual Datapath Schematic



3 Architecturally Visible Registers

The programmer-visible register set of the LIM M2 processor comprises a clean, orthogonal hierarchy:

Register	Width	Architectural Purpose and Features
R0–R7	16-bit	General-Purpose Registers (GPR). Low-latency reading via MUX 1; atomic writeback from the WB_DATA highway under per-register write gating.
SREG	24-bit	Dedicated Stack Register. Supports efficient hardware frame and subroutine stack management with linear 24-bit stack addressing (up to 16 MB).
PTREG	24-bit	High-Precision Pointer Register. Provides flat linear addressing across 16 MB with optional hardware auto-increment.
FREG	16-bit	Status and Control Register. Unifies mathematical ALU condition flags with processor operational control bits.

3.1 Flag and Status Register (FREG) Mapping

The 16-bit FREG provides granular status monitoring and execution mode management:

Bit	Mnemonic	Functional Description
0	ZF	Zero Flag: Asserted when the execution result equals zero.
1	CF	Carry Flag: Captures arithmetic carry/borrow out of the most significant bit.
2	OF	Overflow Flag: Two’s complement signed overflow indicator.
3	EM	Error Math: Hardware trap indicator for mathematical faults (e.g., divide-by-zero).
4–5	CMP	Compare Result: Two-bit hardware comparator status (<, =, >).
6–7	CLK	Clock Mode: Frequency prescaler and core clock domain configuration.
8	PIE	Program Interrupt Enable: Master mask for software interrupts.
9	MIE	Master Interrupt Enable: Hardware external interrupt enable.
10	DWR	Disable Wait for Ready: Forces immediate bus cycles ignoring slave wait states.
11	M24	24-bit Mode Enable: Activates flat 24-bit linear addressing.
12	DBIT	Discarded Bit: Retains the bit ejected during shift operations.
13	SMOD	Shift Mode: Selects shift fill mode for BSL/BSR operations.
14	SBIT	Shift Bit Value: Constant value injected during shifts when SMOD=1.
15	AIP	Auto-Increment PTREG: Automatic hardware increment of PTREG upon memory access.

4 Machine Instruction Format

LIM M2 instructions follow an optimized, dense encoding format. A 5-bit primary opcode dispatches directly into the microcode sequencer:



The 3-bit cfg field selects up to 8 orthogonal execution modes per opcode, enabling expressive addressing and operand variations without opcode proliferation.

5 Instruction Set Architecture Summary

No.	Code	Mnemonic	Class	Functional Summary
0	00000	NOP	Control	No operation and ALU residual chain flush
1	00001	ADD	Arithmetic	Addition with automatic multi-word carry chaining
2	00010	SUB	Arithmetic	Subtraction with automatic multi-word borrow chaining
3	00011	MUL	Arithmetic	Integer multiplication with upper word residual latching
4	00100	DIV	Arithmetic	Integer division with remainder latched in ALU residual
5	00101	LMR	Memory	Load memory word into register
6	00110	LRM	Memory	Store register word into memory
7	00111	LRR	Transfer	Low-latency register-to-register copy via bypass MUX
8	01000	CMP	Compare	Arithmetic compare without modifying operand registers
9	01001	JMP	Branch	Unconditional branch (direct, indirect, 24-bit pointer)
10	01010	JCR	Branch	Conditional branch on Carry Flag (CF)
11	01011	JNZ	Branch	Conditional branch on Non-Zero (ZF = 0)
12	01100	JZ	Branch	Conditional branch on Zero (ZF = 1)
13	01101	AND	Logic	Bitwise logical AND
14	01110	OR	Logic	Bitwise logical OR
15	01111	NAND	Logic	Bitwise logical NAND
16	10000	NOR	Logic	Bitwise logical NOR
17	10001	NOT	Logic	Bitwise bit inversion (one's complement)
18	10010	XOR	Logic	Bitwise exclusive OR
19	10011	XNOR	Logic	Bitwise equivalence
20	10100	INC	Arithmetic	Atomic increment operand by 1
21	10101	DEC	Arithmetic	Atomic decrement operand by 1
22	10110	PUSH	Stack	Push word onto execution stack
23	10111	POP	Stack	Pop word from execution stack into destination
24	11000	BSL	Shift	Barrel shift left with configurable fill
25	11001	BSR	Shift	Barrel shift right with discarded bit capture
26	11010	CSRM	System	Privileged control and status register access
27	11011	RSV27	Reserved	Reserved for vector/SIMD processing extensions
28	11100	RSV28	Reserved	Reserved for extended addressing modes
29	11101	RSV29	Reserved	Reserved for floating-point coprocessor
30	11110	RSV30	Reserved	Reserved for hardware cryptographic acceleration
31	11111	RSV31	Reserved	Reserved for future architectural expansion

6 Detailed Instruction Specifications

The following pages detail the formal execution profiles for all 32 opcode allocations.

NOP

No Operation and Arithmetic Chain Flush

00000
Control

Mnemonic	NOP
Opcode	00000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Control
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

No Operation and Arithmetic Chain Flush.

Architectural Semantics

```
PC <- PC + instruction_length
ALU_RESIDUAL <- 0
FREG.CF <- 0
FREG.OF <- 0
```

Configuration and Operands

No operands required; cfg field reserved for delay calibration.

Flags Affected (FREG)

Clears carry (CF), overflow (OF), and borrow tracking flags; preserves other flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Performs pipeline cycle alignment and flushes the internal ALU residual accumulator, terminating any chained multi-word carry/borrow sequence before starting new independent calculations.

TBD markers denote configurable hardware options.

ADD

Addition with Automatic Multi-Word Carry Chaining

00001
Arithmetic

Mnemonic	ADD
Opcode	00001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Addition with Automatic Multi-Word Carry Chaining.

Architectural Semantics

```
sum_full <- srcA + srcB + ALU_RESIDUAL
dst <- sum_full[15:0]
ALU_RESIDUAL <- (sum_full > 0xFFFF ? 1 : 0)
```

Configuration and Operands

GPRs R0–R7, accumulators ACCA/ACCB, or immediate operands.

Flags Affected (FREG)

Sets carry flag CF and overflow OF if sum exceeds $2^{16} - 1$; updates ZF based on dst.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Delivers seamless multi-word arithmetic (32, 48, 64+ bits). If the addition exceeds 16 bits ($> 2^{16} - 1$), the carry (+1) is latched into the internal ALU residual register and automatically added during

the subsequent ADD operation without requiring distinct ADC opcodes. The lower 16 bits are deterministically stored in dst. The chain is reset via NOP or overwritten by a new operation.

SUB

Subtraction with Automatic Multi-Word Borrow Chaining

00010
Arithmetic

Mnemonic	SUB
Opcode	00010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Subtraction with Automatic Multi-Word Borrow Chaining.

Architectural Semantics

```
diff_full <- srcA - srcB - ALU_RESIDUAL
dst <- diff_full[15:0]
ALU_RESIDUAL <- (diff_full < 0 ? 1 : 0)
```

Configuration and Operands

GPRs R0–R7, constants, or accumulators ACCA/ACCB.

Flags Affected (FREG)

Sets borrow flag (CF) and OF on negative result (< 0); updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Enables continuous multi-word subtraction. When a borrow occurs (< 0), the borrow (–1) is retained in the internal ALU register and automatically subtracted during the next SUB operation. The lower

16-bit word is deterministically preserved in dst. Inserting a NOP breaks the chain for independent subtraction.

TBD markers denote configurable hardware options.

MUL

Multiplication with Upper Word Residual Retention

00011
Arithmetic

Mnemonic	MUL
Opcode	00011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Multiplication with Upper Word Residual Retention.

Architectural Semantics

```
prod <- srcA * srcB
dst <- prod[15:0]
ALU_RESIDUAL <- prod[31:16]
```

Configuration and Operands

GPRs R0–R7, accumulators ACCA/ACCB.

Flags Affected (FREG)

Updates ZF; sets OF/CF if the upper 16 bits of the product (prod[31:16]) are non-zero.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

For 32-bit products exceeding the 16-bit register width, the low 16 bits are placed into dst, while the upper 16 bits are retained in the internal ALU residual register. In a subsequent addition, this

residual is automatically accumulated (hardware MAC support / bigint multiplication). The upper word can also be extracted by adding the register to itself after clearing. Executing NOP flushes the residual register.

DIV

Division with Remainder Latched in ALU Residual

00100
Arithmetic

Mnemonic	DIV
Opcode	00100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Division with Remainder Latched in ALU Residual.

Architectural Semantics

```
dst <- srcA / srcB
ALU_RESIDUAL <- srcA % srcB
```

Configuration and Operands

GPRs R0–R7, accumulators ACCA/ACCB.

Flags Affected (FREG)

Sets EM (Error Math) flag on division by zero without core stall; updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

Stores the integer quotient in dst while preserving the remainder (modulo) in the internal ALU residual register for immediate reuse in subsequent operations or clearing via NOP.

TBD markers denote configurable hardware options.

LMR

Load Word from Memory to Register

00101
Memory

Mnemonic	LMR
Opcode	00101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Memory
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Load Word from Memory to Register.

Architectural Semantics

```
dst_reg <- MEM[address]
```

Configuration and Operands

Addressing: direct, indirect via PTREG, or base-plus-offset.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Supports automated hardware pointer increment when AIP is set.

TBD markers denote configurable hardware options.

LRM

Store Register Word to Memory

00110
Memory

Mnemonic	LRM
Opcode	00110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Memory
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Store Register Word to Memory.

Architectural Semantics

```
MEM[address] <- src_reg
```

Configuration and Operands

Addressing mode configured via cfg and pointer register PTREG.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Direct transfer from register file to memory bus without intermediate staging.

TBD markers denote configurable hardware options.

LRR

Low-Latency Register-to-Register Copy

00111
Transfer

Mnemonic	LRR
Opcode	00111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Transfer
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Low-Latency Register-to-Register Copy.

Architectural Semantics

```
dst_reg <- src_reg
```

Configuration and Operands

Arguments specify source and destination registers.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Routes via the dedicated MUX 1 → MUX 2 bypass, leaving the ALU unblocked.

TBD markers denote configurable hardware options.

CMP

Hardware Operand Comparison

01000
Compare

Mnemonic	CMP
Opcode	01000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Compare
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Hardware Operand Comparison.

Architectural Semantics

```
FREG.CMP <- compare(srcA, srcB)
```

Configuration and Operands

Register or immediate comparison modes.

Flags Affected (FREG)

Updates 2-bit FREG.CMP field and flags ZF, CF, OF without modifying registers.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Enables swift condition evaluation without data corruption.

TBD markers denote configurable hardware options.

JMP

Unconditional Program Branch

01001
Branch

Mnemonic	JMP
Opcode	01001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Unconditional Program Branch.

Architectural Semantics

PC <- target

Configuration and Operands

Supports 16-bit relative, direct absolute, or full 24-bit jump via PTREG.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Atomic reload of the Program Counter in minimal clock cycles.

TBD markers denote configurable hardware options.

JCR

Conditional Branch on Carry

01010
Branch

Mnemonic	JCR
Opcode	01010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Carry.

Architectural Semantics

```
if FREG.CF == 1 then PC <- target
```

Configuration and Operands

Target specified via relative displacement or absolute address.

Flags Affected (FREG)

Flags read without modification.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Foundational primitive for multi-word extended precision arithmetic loops.

TBD markers denote configurable hardware options.

JNZ

Conditional Branch on Non-Zero

01011
Branch

Mnemonic	JNZ
Opcode	01011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Non-Zero.

Architectural Semantics

```
if FREG.ZF == 0 then PC <- target
```

Configuration and Operands

High-efficiency branch for loop counters and iterative structures.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Fast-path condition evaluation in microcode.

TBD markers denote configurable hardware options.

JZ

Conditional Branch on Zero

01100
Branch

Mnemonic	JZ
Opcode	01100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Zero.

Architectural Semantics

```
if FREG.ZF == 1 then PC <- target
```

Configuration and Operands

Evaluates termination or equality conditions.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Optimized for string matching and search algorithms.

TBD markers denote configurable hardware options.

AND

Bitwise Logical AND

01101

Logic

Mnemonic	AND
Opcode	01101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical AND.

Architectural Semantics

```
dst <- srcA AND srcB
```

Configuration and Operands

Register operands or immediate bitmasks.

Flags Affected (FREG)

Updates Zero Flag (ZF); resets overflow status.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Essential for bitmask extraction and masking.

TBD markers denote configurable hardware options.

OR

Bitwise Logical OR

01110
Logic

Mnemonic	OR
Opcode	01110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical OR.

Architectural Semantics

```
dst <- srcA OR srcB
```

Configuration and Operands

Register operands or bitfield insertion.

Flags Affected (FREG)

Updates ZF; carry/overflow deterministically cleared.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Rapid combination of control bitfields.

TBD markers denote configurable hardware options.

NAND

Bitwise Logical NAND

01111
Logic

Mnemonic	NAND
Opcode	01111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical NAND.

Architectural Semantics

```
dst <- NOT (srcA AND srcB)
```

Configuration and Operands

Universal logic primitive.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Enables compact implementation of custom Boolean functions.

TBD markers denote configurable hardware options.

NOR

Bitwise Logical NOR

10000
Logic

Mnemonic	NOR
Opcode	10000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical NOR.

Architectural Semantics

```
dst <- NOT (srcA OR srcB)
```

Configuration and Operands

Inverted logic primitive.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Broadens fast bitwise logic transformations.

TBD markers denote configurable hardware options.

NOT

Bitwise Inversion (One's Complement)

10001
Logic

Mnemonic	NOT
Opcode	10001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Inversion (One's Complement).

Architectural Semantics

```
dst <- NOT src
```

Configuration and Operands

High-speed unary inversion.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Executes in minimal microcycles.

TBD markers denote configurable hardware options.

XOR

Bitwise Exclusive OR

10010
Logic

Mnemonic	XOR
Opcode	10010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Exclusive OR.

Architectural Semantics

```
dst <- srcA XOR srcB
```

Configuration and Operands

Used for bit flipping and rapid register zeroing.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Foundational for parity, CRC, and cryptographic ciphers.

TBD markers denote configurable hardware options.

XNOR

Bitwise Equivalence

10011

Logic

Mnemonic	XNOR
Opcode	10011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Equivalence.

Architectural Semantics

```
dst <- NOT (srcA XOR srcB)
```

Configuration and Operands

Bit-by-bit identity testing.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Fast parallel mask equivalence validation.

TBD markers denote configurable hardware options.

INC

Atomic Increment by 1

10100
Arithmetic

Mnemonic	INC
Opcode	10100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Atomic Increment by 1.

Architectural Semantics

```
dst <- src + 1
```

Configuration and Operands

Fast loop iterator step.

Flags Affected (FREG)

Updates ZF, OF; Carry Flag preserved or updated based on cfg.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Optimizes array index advancing.

TBD markers denote configurable hardware options.

DEC

Atomic Decrement by 1

10101
Arithmetic

Mnemonic	DEC
Opcode	10101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Atomic Decrement by 1.

Architectural Semantics

```
dst <- src - 1
```

Configuration and Operands

Fast downward counter step.

Flags Affected (FREG)

Updates ZF, OF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Compact loop termination construct.

TBD markers denote configurable hardware options.

PUSH

Push Word onto Execution Stack

10110
Stack

Mnemonic	PUSH
Opcode	10110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Stack
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Push Word onto Execution Stack.

Architectural Semantics

```
stack <- src
SREG <- SREG - 2
```

Configuration and Operands

Source: any GPR (R0–R7) or special status register.

Flags Affected (FREG)

Preserves arithmetic flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Hardware-assisted subroutine and context preservation.

TBD markers denote configurable hardware options.

POP

Pop Word from Execution Stack

10111
Stack

Mnemonic	POP
Opcode	10111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Stack
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Pop Word from Execution Stack.

Architectural Semantics

```
dst <- stack
SREG <- SREG + 2
```

Configuration and Operands

Destination: any GPR or status register.

Flags Affected (FREG)

Preserves arithmetic flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Symmetric restoration of execution contexts.

TBD markers denote configurable hardware options.

BSL

Barrel Shift Left

11000

Shift

Mnemonic	BSL
Opcode	11000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Shift
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Barrel Shift Left.

Architectural Semantics

dst <- src « amount

Configuration and Operands

Shift count: immediate constant or dynamic register value.

Flags Affected (FREG)

Ejected MSB latched into FREG.DBIT; supports SMOD/SBIT insertion.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

High-speed arithmetic scaling and bitfield alignment.

TBD markers denote configurable hardware options.

BSR

Barrel Shift Right

11001

Shift

Mnemonic	BSR
Opcode	11001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Shift
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Barrel Shift Right.

Architectural Semantics

`dst <- src » amount`

Configuration and Operands

Logical or sign-extended arithmetic right shift selectable via cfg.

Flags Affected (FREG)

Discarded LSB latched into FREG.DBIT.

Datapath Micro-Operations

- 1. Multiplexer selection of source operands from register file or immediate bus.
- 2. Operation dispatch to dedicated execution block (ALU / memory / control).
- 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
- 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Fast scaling and power-of-two division.

TBD markers denote configurable hardware options.

CSRM

Control and Status Register Access

11010
System

Mnemonic	CSRM
Opcode	11010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	System
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Control and Status Register Access.

Architectural Semantics

```
special_reg <- src OR dst <- special_reg
```

Configuration and Operands

Access to FREG, PTREG, and SREG.

Flags Affected (FREG)

Flags updated when FREG is modified.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Privileged context switching and configuration primitive.

TBD markers denote configurable hardware options.

RSV27

Architectural Extension Slot

11011
Reserved

Mnemonic	RSV27
Opcode	11011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

reserved

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV28

Architectural Extension Slot

11100
Reserved

Mnemonic	RSV28
Opcode	11100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

reserved

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV29

Architectural Extension Slot

11101
Reserved

Mnemonic	RSV29
Opcode	11101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

reserved

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV30

Architectural Extension Slot

11110
Reserved

Mnemonic	RSV30
Opcode	11110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

reserved

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV31

Architectural Extension Slot

11111
Reserved

Mnemonic	RSV31
Opcode	11111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

reserved

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

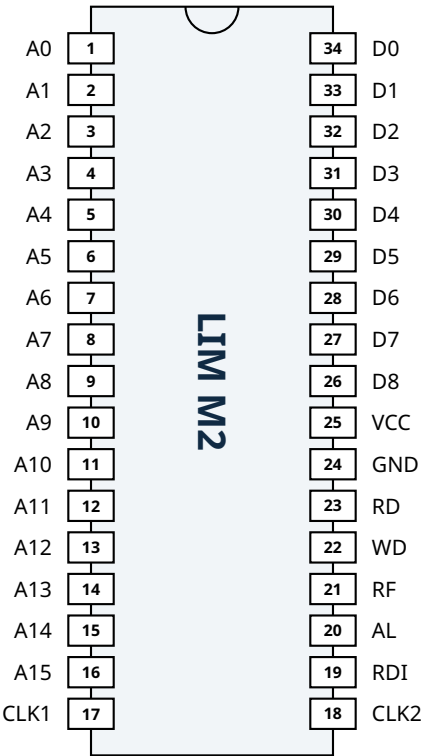
Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

7 Physical Package and Pinout (34-Pin Form Factor)

The processor is packaged in a high-reliability dual-inline 34-pin enclosure:



7.1 Pin Descriptions

Signal	Direction	Functional Role
A0–A15	Output / Tri-State	16-bit primary address bus (latched via external AL strobe).
D0–D7	Bidirectional	8-bit external memory and peripheral data interface.
CLK1, CLK2	Input	Two-phase non-overlapping clock synchronization inputs.
VCC, GND	Power	Primary power supply rails (+5 V nominal).
RD, WD	Output	Bus Read Enable and Write Enable strobes.
AL	Output	Address Latch strobe for external high-speed address registers.
RF	Input	Ready Flag: external slave memory handshake acknowledge.
RDI	Input	Raw Direct Interrupt: priority hardware interrupt request line.

8 Conclusion

The **LIM M2** architecture harmonizes the time-tested reliability of deterministic microprogramming with the modern performance of an entirely tri-state-free multiplexer datapath. By eliminating bus contention, floating nodes, and capacitive decay, the LIM M2 guarantees exceptional signal fidelity and opens an unprecedented clock frequency envelope spanning from 200 MHz to the highest thresholds of modern silicon.